

首先建立一个临时关系 K :

K	
Cno	
81001	
81003	

然后求:

$$\Pi_{Sno, Cno}(SC) \div K = \{20180001, 20180002\}$$

求解过程与例 2.9 类似, 先对关系 SC 在 (Sno, Cno) 属性上投影, 然后逐一求出每一学生 (Sno) 的象集, 并依次检查这些象集是否包含 K 。

[例 2.11] 查询选修了 81002 号课程的学生的学号。

$$\Pi_{Sno}(\sigma_{Cno='81002'}(SC)) = \{20180001, 20180002, 20180003\}$$

[例 2.12] 查询至少选修了一门其直接先修课为 81003 号课程的学生的姓名。

$$\Pi_{Sname}(\sigma_{Cpno='81003'}(Course) \bowtie SC \bowtie \Pi_{Sno, Sname}(Student))$$

或

$$\Pi_{Sname}(\Pi_{Sno}(\sigma_{Cpno='81003'}(Course) \bowtie SC) \bowtie \Pi_{Sno, Sname}(Student))$$

[例 2.13] 查询选修了全部课程的学生的学号和姓名。

$$\Pi_{Sno, Cno}(SC) \div \Pi_{Cno}(Course) \bowtie \Pi_{Sno, Sname}(Student)$$

关系代数小结

本节介绍了 8 种关系代数运算, 其中并、差、笛卡儿积、选择和投影这 5 种运算为基本的运算。其他三种运算, 即交、连接和除, 均可以用这 5 种基本运算来表达。引进它们并不增加语言的运算能力, 但可以简化表达。

关系代数中, 这些运算经有限次复合而形成的表达式称为关系代数表达式。

此外, 还有扩展的关系代数, 如关系的重新命名、查询结果的去重操作、分组操作、排序操作和聚集函数等, 这里就不介绍了。

* 2.5 关系演算

关系演算是以数理逻辑中的谓词演算为基础的。按谓词变元不同, 关系演算可分为元组关系演算和域关系演算。本节通过元组关系演算语言 ALPHA 和域关系演算语言 QBE 来介绍关系演算的基本概念和查询方法。

* 2.5.1 元组关系演算语言 ALPHA

元组关系演算以元组变量作为谓词变元的基本对象。一种典型的元组关系演算语言是 Edgar F. Codd 提出的 ALPHA。这一语言虽然没有实际实现, 但关系数据库管理系统 INGRES 最初所用的 QUEL 语言就是参照 ALPHA 研制的, 与其十分类似。

ALPHA 语言主要有 GET、PUT、HOLD、UPDATE、DELETE、DROP 这 6 条语句。其语句的基本格式如下。

操作语句 工作空间名(表达式): 操作条件

其中,“表达式”用于指定语句的操作对象,它可以是关系名或(和)属性名,一条语句可以同时操作多个关系或多个属性。“操作条件”是一个逻辑表达式,用于将操作结果限定在满足条件的元组中,操作条件可以为空。除此之外,还可以在基本格式的基础上加上排序要求以及指定返回元组的条数等。

1. 检索操作

检索操作用 GET 语句实现。

(1) 简单检索(即不带条件的检索)

[例 2.14] 查询所有被选修的课程的课程号。

```
GET W(SC. Cno)
```

其中,“W”为工作空间名。这里条件为空,表示没有限定条件。

[例 2.15] 查询所有学生的数据。

```
GET W(Student)
```

(2) 限定的检索(即带条件的检索)

[例 2.16] 查询计算机科学与技术专业 2000 年之前出生的学生的学号和出生日期。

```
GET W(Student. Sno, Student. Sbirthdate): Student. Smajor='计算机科学与技术'  
      ^ Student. Sbirthdate<'2000-1-1'
```

(3) 带排序的检索

[例 2.17] 查询计算机科学与技术专业学生的学号和出生日期,结果按出生日期降序排序。

```
GET W(Student. Sno, Student. Sbirthdate): StudentSmajo='计算机科学与技术'  
      DOWN Student. Sbirthdate
```

其中,“DOWN”表示降序排序。

(4) 指定返回元组的条数的检索

[例 2.18] 取出一个信息安全专业学生的学号。

```
GET W(1)(Student. Sno): Student. Smajor='信息安全'
```

其中,“W”后括号中的数量就是指定的返回元组的个数。

[例 2.19] 查询选修了 81003 号课程、成绩在前三名的学生的学号及其成绩。

```
GET W(3)(SC. Sno, SC. Grade): SC. Cno='81003' DOWN SC. Grade
```

(5) 用元组变量的检索

前面已讲到,元组关系演算是以元组变量作为谓词变元的基本对象。元组变量是在某一关

系范围内变化的,所以也称为范围变量(range variable),一个关系可以设多个元组变量。

元组变量主要有两方面的用途:

① 简化关系名。如果关系名很长,使用起来感到不方便,则可以设一个较短名字的元组变量来代替关系名。

② 操作条件中使用量词时必须用元组变量。

[例 2.20] 查询信息安全专业学生的姓名。

```
RANGE Student X
```

```
GET W(X.Sname):X.Smajor='信息安全'
```

ALPHA 语言用 RANGE 来说明元组变量。本例中 X 是关系 Student 上的元组变量,用途是简化关系名,即用 X 代表 Student。

(6) 用存在量词(existential quantifier)的检索

例 2.21、例 2.22、例 2.23 中的元组变量都是为存在量词而设的。其中例 2.23 需要对两个关系使用存在量词,所以设了两个元组变量。

[例 2.21] 查询选修了 81002 号课程的学生的姓名。

```
RANGE SC X
```

```
GET W(Student.Sname): $\exists X(X.Sno=Student.Sno \wedge X.Cno='81002')$ 
```

[例 2.22] 查询选修了直接先修课程是 81002 号课程的学生的学号。

```
RANGE Course CX
```

```
GET W(SC.Sno): $\exists CX(CX.Cno=SC.Cno \wedge CX.Cpno='81002')$ 
```

[例 2.23] 查询至少选修一门其先修课为 81002 号课程的学生的姓名。

```
RANGE Course CX
```

```
SC SCX
```

```
GET W(Student.Sname): $\exists SCX(SCX.Sno=Student.Sno \wedge$   
 $\exists CX(CX.Cno=SCX.Cno \wedge CX.Cpno='81002'))$ 
```

在本例中的元组关系演算公式可以变换为前束范式(prenex normal form)的形式:

```
RANGE Course CX
```

```
SC SCX
```

```
GET W(Student.Sname): $\exists SCX \exists CX(SCX.Sno=Student.Sno \wedge$   
 $CX.Cno=SCX.Cno \wedge CX.Pcno='81002')$ 
```

(7) 带有多个关系的表达式的检索

上面所举的各个例子中,虽然查询时可能会涉及多个关系,即公式中可能涉及多个关系,但查询结果表达式中只有一个关系。实际上表达式中是可以有多个关系的。

[例 2.24] 查询成绩为 90 分以上的学生姓名与课程名称。

本查询所要求的结果是学生姓名和课程名称,分别在 Student 和 Course 两个关系中。

RANGE SC SCX

GET W(Student. Sname, Course. Cname) : $\exists$ SCX(SCX. Grade $\geq 90 \wedge$
SCX. Sno = Student. Sno $\wedge$ Course. Cno = SCX. Cno)

(8) 用全称量词(universal quantifier)的检索

[例 2.25] 查询不选 81004 号课程的学生姓名。

RANGE SC SCX

GET W(Student. Sname) : $\forall$ SCX(SCX. Sno $\neq$ Student. Sno $\vee$ SCX. Cno $\neq$ '81004')

本例也可以用存在量词来表示:

RANGE SC SCX

GET W(Student. Sname) : $\neg \exists$ SCX(SCX. Sno = Student. Sno $\wedge$ SCX. Cno = '81004')

(9) 用两种量词的检索

[例 2.26] 查询选修了全部课程的学生姓名。

RANGE Course CX

SC SCX

GET W(Student. Sname) : $\forall$ CX $\exists$ SCX(SCX. Sno = Student. Sno $\wedge$ SCX. Cno = CX. Cno)

(10) 用蕴涵(implication)的检索

[例 2.27] 查询至少选修了 20180003 号学生所选课程的学生学号。

本例题的求解思路是, 对 Course 中的所有课程依次检查每一门课程, 看 20180003 是否选修了该课程, 如果选修了, 则再看某一个学生是否也选修了该门课。如果对于 20180003 所选的每门课程该学生都选修了, 则该学生为满足要求的学生。把所有这样的学生全都找出来即完成了本题。

RANGE Course CX

SC SCX /* 注意, 这里 SC 设了两个元组变量 */

SC SCY

GET W(Student. Sno) : $\forall$ CX($\exists$ SCX (SCX. Sno = '20180003' $\wedge$ SCX. Cno = CX. Cno)
 $\Rightarrow \exists$ SCY(SCY. Sno = Student. Sno $\wedge$ SCY. Cno = CX. Cno))

(11) 聚集函数

用户在使用查询语言时经常要做一些简单的计算, 例如要计算符合某一查询要求的元组数, 计算某个关系中所有元组在某属性上的值的总和或平均值等。为了方便用户, 关系数据语言中建立了有关这类运算的标准函数库供用户选用。这类函数通常称为聚集函数或内置函数(built-in function)。关系演算中提供了 COUNT、TOTAL、MAX、MIN、AVG 等聚集函数, 其含义如表 2.6 所示。

表 2.6 关系演算中的聚集函数

函数名	功能
COUNT	对元组计数
TOTAL	求总和
MAX	求最大值
MIN	求最小值
AVG	求平均值

[例 2.28] 查询学生所选的专业的数目。

```
GET W( COUNT( Student. Smajor ) )
```

COUNT 函数在计数时会自动排除重复值。

[例 2.29] 查询 2020 年第二学期选修了 81003 号课程的学生的平均成绩。

```
GET W( AVG( SC. Grade ) ) : SC. Cno = '81003' ^ SC. Semester = '20202'
```

2. 更新操作

(1) 修改操作

修改操作用 UPDATE 语句实现。其步骤是：

- ① 用 HOLD 语句将要修改的元组从数据库读到工作空间中。
- ② 用宿主语言修改工作空间中元组的属性值。
- ③ 用 UPDATE 语句将修改后的元组送回数据库。

需要注意的是，单纯检索数据使用 GET 语句即可，但为修改数据而读元组时必须使用 HOLD 语句，HOLD 语句是带并发控制的 GET 语句。有关并发控制的概念将在第 12 章并发控制中详细介绍。

[例 2.30] 把学号为 20180004 的学生从计算机科学与技术专业转到信息管理与信息系统专业。

```
HOLD W( Student. Sno, Student Smajor ) : Student. Sno = '20180004'
/* 从 Student 关系中读出学号为 20180004 的学生的数据 */
MOVE '信息管理与信息系统' TO W. Smajor /* 用宿主语言进行修改 */
UPDATE W /* 把修改后的元组送回 Student 关系 */
```

在该例中用 HOLD 语句来读 20180004 号学生的数据，而不是用 GET 语句。

如果修改操作涉及两个关系，就要执行两次 HOLD-MOVE-UPDATE 操作序列。

在 ALPHA 语言中，修改关系主码的操作是不允许的，例如不能用 UPDATE 语句将学号“20180004”改为“20180009”。如果需要修改主码值，只能先用删除操作删除该元组，然后再把具有新主码值的元组插入关系。

(2) 插入操作

插入操作用 PUT 语句实现。其步骤是：

- ① 用宿主语言在工作空间中建立新元组。
- ② 用 PUT 语句将该元组插入指定的关系。

[例 2.31] 学校新开设了一门 2 学分的课程“计算机组织与结构”，其课程号为 81009，直接先行课为 81005 号课程。插入该课程元组。

```
MOVE '81009' TO W. Cno
MOVE '计算机组织与结构' TO W. Cname
```

```
MOVE '2' TO W. Ccredit  
MOVE '81005' TO W. Cpno  
PUT W( Course)      /* 把 W 中的元组插入指定关系 Course */
```

PUT 语句只对一个关系操作,也就是说表达式必须为单个关系名。

(3) 删除

删除操作用 DELETE 语句实现。其步骤为:

- ① 用 HOLD 语句把要删除的元组从数据库读到工作空间中。
- ② 用 DELETE 语句删除该元组。

[例 2.32] 20180007 号学生因故退学,删除该学生元组。

```
HOLD W( Student) : Student. Sno = '20180007'  
DELETE W
```

[例 2.33] 将学号 20180001 改为 20180010。

```
HOLD W( Student) : Student. Sno = '20180001'  
DELETE W  
MOVE '20180010' TO W. Sno  
MOVE '李勇' TO W. Sname  
MOVE '男' TO W. Ssex  
MOVE '2000-3-8' TO W. Sbirthdate  
MOVE '信息安全' TO W. Smajor  
PUT W( Student)
```

[例 2.34] 删除全部学生。

```
HOLD W( Student)  
DELETE W
```

由于 SC 关系与 Student 关系之间具有参照关系,为保证参照完整性,删除 Student 中的元组时要相应地删除 SC 中的元组(手工删除或由数据库管理系统自动删除)。

```
HOLD W( SC)  
DELETE W
```

ALPHA 语言例题解析可参见二维码内容。



扩展阅读: ALPHA
语言例题解析

* 2.5.2 域关系演算语言 QBE

关系演算的另一种形式是域关系演算。域关系演算以元组变量的分量(即域变量)作为谓词变元的基本对象。1975 年由 M. M. Zloof 提出的 QBE 就是一个很有特色的域关系演算语言,

该语言于 1978 年在 IBM 370 上得以实现。

QBE 是 Query By Example(即通过例子进行查询)的简称,它最突出的特点是操作方式。它是一种高度非过程化的基于屏幕表格的查询语言,用户通过终端屏幕编辑程序,以填写表格的方式构造查询要求,而查询结果也是以表格形式显示,因此非常直观、易学易用。

QBE 用示例元素来表示查询结果可能的情况,示例元素实质上就是域变量。QBE 的操作框架如图 2.12 所示。

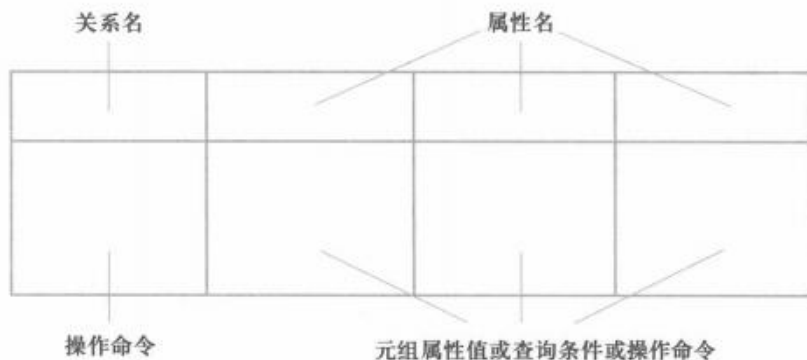


图 2.12 QBE 的操作框架

下面仍以 2.1.3 小节图 2.1 所示的“学生选课”关系数据库为例,说明 QBE 的用法。

1. 检索操作

(1) 简单查询

[例 2.35]求信息安全专业全体学生的姓名。

操作步骤如下:

- ① 用户提出要求。
- ② 屏幕显示如下空白表格。

- ③ 用户在最左边一栏输入关系名 Student。

Student					

- ④ 系统显示该关系的属性名。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor

⑤ 用户在上面构造查询要求。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. <u>T</u>			信息安全

这里“T”是示例元素，即域变量。QBE 要求示例元素下面一定要加下划线。“信息安全”是查询条件，不用加下划线。“P.”是操作符，表示打印(print)，实际上是显示。

查询条件中可以使用比较运算符>、≥、<、≤、=和≠，其中=可以省略。

示例元素是这个域中可能的一个值，它不必是查询结果中的元素。比如若求信息系的学生，只要给出任意的一个学生名即可，而不必真是信息系的某个学生名。

对于例 2.35，可构造查询要求如下：

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. <u>刘晨</u>			信息安全

这里的查询条件是 Smajor='信息安全'，其中“=”被省略。

⑥ 屏幕显示查询结果。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		<u>李勇</u>			信息安全

即根据用户的查询要求找出信息系的学生姓名。

[例 2.36] 查询全体学生的全部数据。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>20180002</u>	P. <u>刘晨</u>	P. <u>女</u>	P. <u>1999-9-1</u>	P. <u>计算机科学与技术</u>

显示全部数据也可以简单地把 P. 操作符作用在关系名上。因此本查询也可以简单地表示如下：

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
P.					

(2) 条件查询

[例 2.37] 求出生日期晚于 1999-9-1 的学生的学号。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>20180002</u>			>1999-9-1	

[例 2.38] 求计算机科学与技术专业出生日期晚于 1999-9-1 的学生的学号。

本查询的条件是 Smajor='计算机科学与技术'和 Sbirthdate>'1999-9-1'两个条件的“与”。在 QBE 中，表示两个条件的“与”有两种方法：

① 把两个条件写在同一行上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002			>1999-9-1	计算机科学与技术

② 把两个条件写在不同行上，但使用相同的示例元素值。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002				计算机科学与技术
	P. 20180002			>1999-9-1	

[例 2.39] 查询计算机科学与技术专业或者出生日期晚于 1999-9-1 的学生的学号。

本查询的条件是 Smajor='计算机科学与技术'和 Sbirthdate>'1999-9-1'两个条件的“或”。在 QBE 中把两个条件写在不同行上，并且使用不同的示例元素值，即表示条件的“或”。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. 20180002				计算机科学与技术
	P. 20180003			>1999-9-1	

对于多行条件的查询，先输入哪一行是任意的，不影响查询结果。这就允许用户以不同的思考方式进行查询，十分灵活、自由。

[例 2.40] 查询既选修了 81001 号课程又选修了 81002 号课程的学生的学号。

本查询条件是在一个属性中的“与”关系，它只能用“与”条件的方法②表示，即写两行，但示例元素相同。

SC	Sno	Cno	Grade	Semester	Teachingclass
	P. 20180002	81001			
	P. 20180002	81002			

[例 2.41] 查询选修 81001 号课程的学生的姓名。

本查询涉及 SC 和 Student 两个关系。在 QBE 中实现这种查询的方法是通过相同的连接属性值把多个关系连接起来。这里示例元素 Sno 是连接属性，其值在两个表中要相同。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	20180002	P. 李勇			

SC	Sno	Cno	Grade	Semester	Teachingclass
	20180002	81001			

[例 2.42] 查询未选修 81001 号课程的学生的姓名。

这里的查询条件中用到逻辑“非”。在 QBE 中表示逻辑“非”的方法是将运算符“¬”写在关系名下面。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	<u>2018002</u>	P. 李勇			
	<u>2018002</u>	P. 李勇			

SC	Sno	Cno	Grade	Semester	Teachingclass
┐	<u>2018002</u>	81001			
┐	<u>2018002</u>				

这个查询就是显示学号示例为“2018002”的学生的姓名，而该学生选修 81001 号课程的情况为假或者什么课程都没有选修。

[例 2.43] 查询有两个人以上选修的课程课程号。

本查询是在一个表内连接。这个查询就是要显示这样的课程号“81001”，它不仅被学生“2018002”选修，而且也被另一个学生“┐ 2018002”选修了。

SC	Sno	Cno	Grade	Semester	Teachingclass
	<u>2018002</u>	P. <u>81001</u>			
	┐ <u>2018002</u>	<u>81001</u>			

(3) 聚集函数

为了方便用户，QBE 提供了一些聚集函数，主要包括 CNT、SUM、AVG、MAX、MIN 等，其含义如表 2.7 所示。

表 2.7 QBE 中的聚集函数

函数名	功能
CNT	对元组计数
SUM	求总和
AVG	求平均值
MAX	求最大值
MIN	求最小值

[例 2.44] 查询信息安全专业学生的总人数。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	P. <u>CNT</u>			.	信息安全

(4) 对查询结果排序

对查询结果按某个属性值的升序排序，只需在相应列中填入“AO.”，按降序排序则填“DO.”。如果按多列排序，用“AO(*i*).”或“DO(*i*).”表示，其中 *i* 为排序的优先级，*i* 值越小，优先级越高。

[例 2.45] 查询全体男生的姓名，要求查询结果按专业名升序排序，对同一专业的学生按出生日期降序排序。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
		P. 李勇	男	DO(2).	AO(1).

2. 更新操作

(1) 修改操作

修改操作符为“U.”。QBE 不允许修改关系的主码，如果需要修改某个元组的主码，只能先删除该元组，然后再插入新主码的元组。

[例 2.46] 把学号为 20180001 的学生的出生日期改为“2001-3-8”。

这是一个简单修改操作，不包含算术表达式，因此可以有两种表示方法：

① 将操作符“U.”放在值上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
	20180001			U. 2001-3-8	

② 将操作符“U.”放在关系上。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
U.	20180001			2001-3-8	

这里，码“20180001”标明要修改的元组。“U.”标明是修改后的新值。由于主码是不能修改的，所以即使在第二种写法中，系统也不会混淆要修改的属性值。

[例 2.47] 将学号为 20180004 的学生在 2019 年第 2 学期选修 81001 号课程的成绩增加 6 分。

这个修改操作涉及表达式，所以只能将操作符“U.”放在关系上。

SC	Sno	Cno	Grade	Semester	Teachingclass
	20180004	81001	56	20192	
U.	20180004		56+6		

[例 2.48] 将 2019 年第 2 学期选修 81001 号课程的学生们的成绩都增加 5 分。

SC	Sno	Cno	Grade	Semester	Teachingclass
	<u>20180001</u>	81001	<u>70</u>	20192	
U.	<u>20180001</u>		<u>70+5</u>	20192	

(2) 插入操作

插入操作符为“I.”。新插入的元组必须具有主码值，其他属性值可以为空。

[例 2.49] 把计算机科学与技术专业的学生“高大卫，男，学号 20190006，出生日期 2001-6-28”存入数据库。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
I.	20190006	高大卫	男	2001-6-28	计算机科学与技术

(3) 删除操作

删除操作符为“D.”。

[例 2.50] 删除学号为 20180006 的学生。

Student	Sno	Sname	Ssex	Sbirthdate	Smajor
D.	20180006				

由于 SC 关系与 Student 关系之间具有参照关系, 为保证参照完整性, 删除学号为 20180006 的学生后, 通常还应删除该学生选修的全部课程。

SC	Sno	Cno	Grade	Semester	Teachingclass
D.	20180006				

本章小结

关系模型是关系数据库的核心概念和基础, 关系数据库系统是目前使用最广泛的数据库系统。20 世纪 70 年代以后开发的数据库管理系统产品几乎都是基于关系模型的。在数据库发展的历史上, 最重要的创新成果之一就是关系模型。

关系模型与层次模型、网状模型最重要的区别是, 关系模型只有“表”这一种数据结构, 而层次模型、网状模型还有其他数据结构, 以及对这些数据结构的操作。关系模型的数据结构简单清晰, 用户容易理解, 又具有坚实的数学基础。



扩展阅读: 元组
关系演算表达式

本章系统地讲解了关系模型的重要概念, 包括关系模型的数据结构、关系操作以及关系的三类完整性约束; 介绍了用代数方式和逻辑方式来表达的关系语言, 即关系代数、元组关系演算和域关系演算。在关系演算中介绍了元组关系演算语言 ALPHA 和域关系演算语言 QBE。有关元组关系演算表达式的内容, 感兴趣的读者可参见二维码介绍。

习 题 2

1. 试述关系模型的三个组成部分。
2. 简述关系数据语言的特点和分类。
3. 定义并理解下列术语, 说明它们之间的联系与区别:
 - ① 域, 笛卡儿积, 关系, 元组, 属性;
 - ② 主码, 全码, 候选码, 外码, 主属性、非主属性;
 - ③ 关系模式, 关系, 关系数据库。
4. 举例说明关系模式和关系的区别。
5. 试述关系模型的完整性约束。在参照完整性中, 什么情况下外码属性的值可以为空值?
6. 设有一个 SPJ 数据库, 包括 4 个关系模式 S、P、J 和 SPJ。

S(SNO, SNAME, STATUS, CITY);

P(PNO, PNAME, COLOR, WEIGHT);

J(JNO,JNAME,CITY);
SPJ(SNO,PNO,JNO,QTY)。

供应商表 S 由供应商代码(SNO)、供应商姓名(SNAME)、供应商状态(STATUS)、供应商所在城市(CITY)组成。

零件表 P 由零件代码(PNO)、零件名(PNAME)、颜色(COLOR)、重量(WEIGHT)组成。

工程项目表 J 由工程项目代码(JNO)、工程项目名(JNAME)、工程项目所在城市(CITY)组成。

供应情况表 SPJ 由供应商代码(SNO)、零件代码(PNO)、工程项目代码(JNO)、供应数量(QTY)组成,表示某供应商供应某种零件给某工程项目的数量为 QTY。

今有若干数据如图 2.13 所示。

S

SNO	SNAME	STATUS	CITY
S1	精益	20	天津
S2	盛锡	10	北京
S3	东方红	30	北京
S4	丰泰盛	20	天津
S5	为民	30	上海

P

PNO	PNAME	COLOR	WEIGHT
P1	螺母	红	12
P2	螺栓	绿	17
P3	螺丝刀	蓝	14
P4	螺丝刀	红	14
P5	凸轮	蓝	40
P6	齿轮	红	30

J

JNO	JNAME	CITY
J1	三建	北京
J2	一汽	长春
J3	弹簧厂	天津
J4	造船厂	天津
J5	机车厂	唐山
J6	无线电厂	常州
J7	半导体厂	南京

SPJ

SNO	PNO	JNO	QTY
S1	P1	J1	200
S1	P1	J3	100
S1	P1	J4	700
S1	P2	J2	100
S2	P3	J1	400
S2	P3	J2	200
S2	P3	J4	500
S2	P3	J5	400
S2	P5	J1	400
S2	P5	J2	100
S3	P1	J1	200
S3	P3	J1	200
S4	P5	J1	100
S4	P6	J3	300
S4	P6	J4	200
S5	P2	J4	100
S5	P3	J1	200
S5	P6	J2	200
S5	P6	J4	500

图 2.13 SPJ 数据库数据

试用关系代数、元组关系演算语言 ALPHA 和域关系演算语言 QBE 完成如下查询:

- ① 求供应工程 J1 零件的供应商代码 SNO。
- ② 求供应工程 J1 零件 P1 的供应商代码 SNO。
- ③ 求供应工程 J1 零件为红色的供应商代码 SNO。
- ④ 求没有使用天津供应商生产的红色零件的工程号 JNO。

- ⑤ 求至少使用了与供应商 S1 所供应的全部零件相同零件号的工程号 JNO。
7. 试述等值连接与自然连接的区别和联系。
8. 关系代数的基本运算有哪些? 如何用这些基本运算来表示其他运算?

参考文献 2

[1] CODD E F. A relational model of data for large shared data banks[J]. CACM-Communications of the ACM, 1970, 13(6): 377-387.

1970 年, Edgar F. Codd 在文献[1]中首先提出了关系数据模型, 以后 Codd 又提出了关系代数和关系演算的概念, 以及函数依赖的概念, 1972 年提出了关系的第一、第二、第三范式, 1974 年提出了 BC(Boyce-Codd) 范式, 为关系数据库系统奠定了理论基础。由于对关系数据库理论的突出贡献, 1981 年 Codd 获得了图灵奖。

[2] CODD E F. A data base sublanguage founded on the relational calculus[C]. Proceedings of the ACM SIGMOD Workshop on Data Description, Access and Control, 1971: 35-68.

[3] CODD E F. Relational completeness of database sublanguages in data base systems[R]. Courant Computer Science Symposia Series, 1972(6).

[4] CODD E F. Further normalization of the data base relational model[M]. Data Base Systems. Prentice-Hall, 1972.

[5] ULLMAN J D. Principles of database systems[M]. 2nd ed. Computer Science Press, 1982.

Ullman 在文献[5]中给出了关系代数、元组关系演算和域关系演算的等价性证明。

[6] ZLOOF M M. Query By Example[C]. Proceeding of the National Computer Conference and Exposition(AFIPS'75), 1975: 431-438.

[7] LACROIX M, PIROTTE A. Domain-oriented relational language. Proceedings of the 3rd International Conference on Very Large Data Bases[C], 1977: 370-378.

[8] LACROIX M, PIROTTE A. ILL: an English structured query language for relational data bases[J]. ACM SIGART Bulletin, 1977, 61: 61-63.

域关系演算的思想出现在文献[7]描述的 QBE 语言中, 形式化定义由 Lacroix 和 Pirotte 在文献[8]中给出。文献[8]给出了一个域关系演算语言 ILL。

[9] STONEBRAKER M, WONG E, KREPS P, et al. The design and implementation of INGRES[J]. ACM Transactions on Database Systems, 1976, 1(3): 189-222.

[10] DATE C J. Referential integrity[C]. Proceedings of the 7th International Conference on Very Large Data Bases, 1981, 7: 2-12.

[11] DATE C J. Why relations should have exactly one primary key[R]. ANSI Database Committee (X3H2) Working Paper X3H2-84-118, 1984.

本章知识点讲解微视频:



关系模型



关系代数



关系演算

除法运算

第3章

关系数据库标准语言 SQL

结构化查询语言(SQL)是关系数据库的标准语言,包括数据查询、数据库模式创建、数据库数据的增删改、数据库安全性和完整性定义与控制等一系列功能。

本章导读

本章详细介绍 SQL 的基本功能,并进一步讲述关系数据库的基本概念,关系数据库管理系统(RDBMS)对数据库三级模式的支持。

本章首先讲述了 SQL 的产生与发展、SQL 的特点,非过程化 SQL 和过程化语言的区别。读者可以从中进一步了解关系数据库技术和关系数据库管理系统产品的发展过程,从而体会关系数据库系统能够减轻用户负担,提高用户开发数据库应用系统生产率的优点。然后通过实例详细讲解 SQL 的功能、语法和使用要点。

SQL 功能丰富、简洁易懂,不过根据需求正确无误地写出 SQL 语句并不容易,特别是使用 SQL 完成复杂查询更需要认真练习才能掌握。

希望读者通过上机实际操作掌握 SQL 的运用,包括数据表定义、数据的查询、插入、删除、更新,以及索引和视图的创建和删除等操作。

本章的难点是正确写出 SQL 语句并完成各种查询,学习方法就是反复训练,通过具体的数据库产品上进行实际练习,才能举一反三牢固掌握。

3.1 SQL 概述

自 SQL 成为数据库国际标准语言以来,几乎所有数据库厂商都采用了 SQL,而且许多软件厂商也纷纷推出与 SQL 接口的软件,使不同数据库系统之间、数据库系统与其他软件系统之间的互操作有了共同的基础。其意义重大,因此有人把确立 SQL 为关系数据库标准语言及其后的发展称为是一场革命。

3.1.1 SQL 的产生与发展

SQL 是在 1974 年由 Boyce 和 Chamberlin 提出的,最初的名称是 SEQUEL(structured English